

AWS API Gateway: API Keys and Usage Plans

To Begin with the Lab

Summary of the Lab

In this lab, an **API key** and a **usage plan** were configured for an existing **REST API** in **API Gateway**. The API already had methods deployed to a stage, so the setup focused on controlling access rather than rebuilding the API itself. A key named **user-api-key** was created, a usage plan named **basic-usage-plan** was created, and the REST API stage **Dev** was attached to that plan. The API key was then attached to the usage plan, and the POST method was marked as **API key required** before redeploying the API. The lab confirmed the behavior in **Postman**: without the **x-api-key** header the request returned **403 Forbidden**, and with the header the request reached **Lambda** successfully. Later, enabling quota caused requests beyond the allowed limit to return **429 Too Many Requests**.

Understanding API Keys and Usage Plans

API keys and **usage plans** exist so you can control and track how clients use your **API Gateway REST API**. The API key identifies the client, while the usage plan defines which API stage the key can access and can also enforce throttling or quotas. This helps protect the backend, limit abuse, and manage access at the client level. In this lab, the method-level setting **API Key Required = True** made the POST route reject requests that did not include the required header.

Example:

A client sends a POST /user request to create a new user record. If the request does not include **x-api-key**, API Gateway blocks it with **403 Forbidden**. When the same request includes the valid key, the request is accepted and reaches the Lambda function that writes the record to **DynamoDB**.

Lab Steps

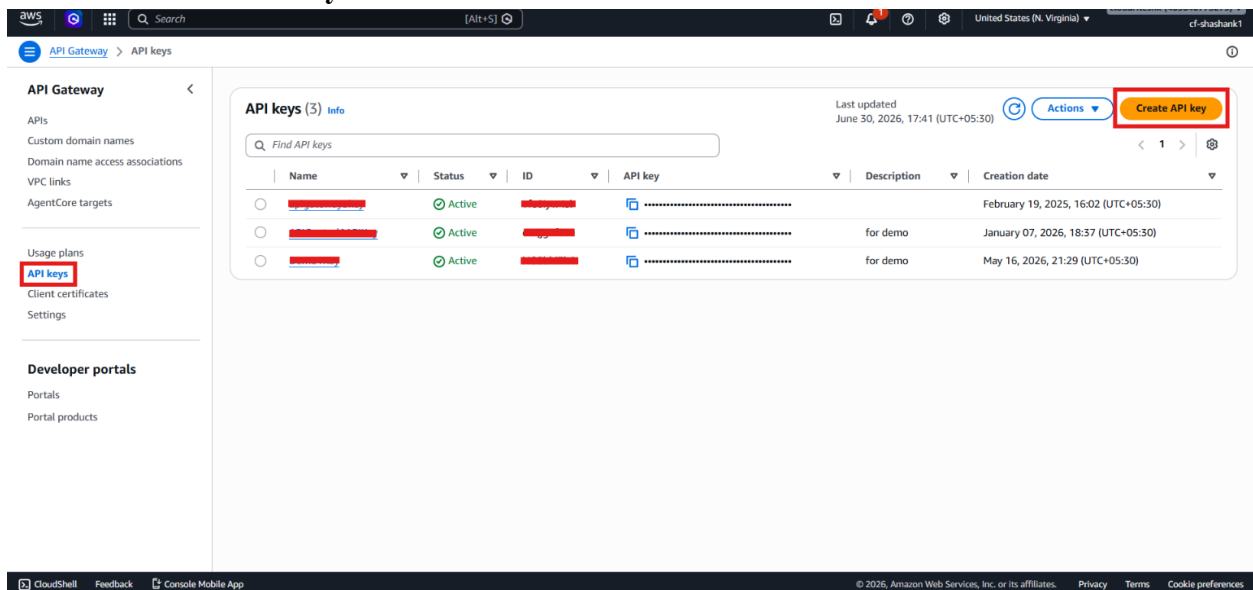
Step 1 – Confirm the Prerequisites

- Sign in to the AWS Management Console.
- Open **API Gateway**.
- Confirm that the **REST API** already exists.
- Confirm that the methods (GET, POST, DELETE) are already deployed to a stage such as **dev**.

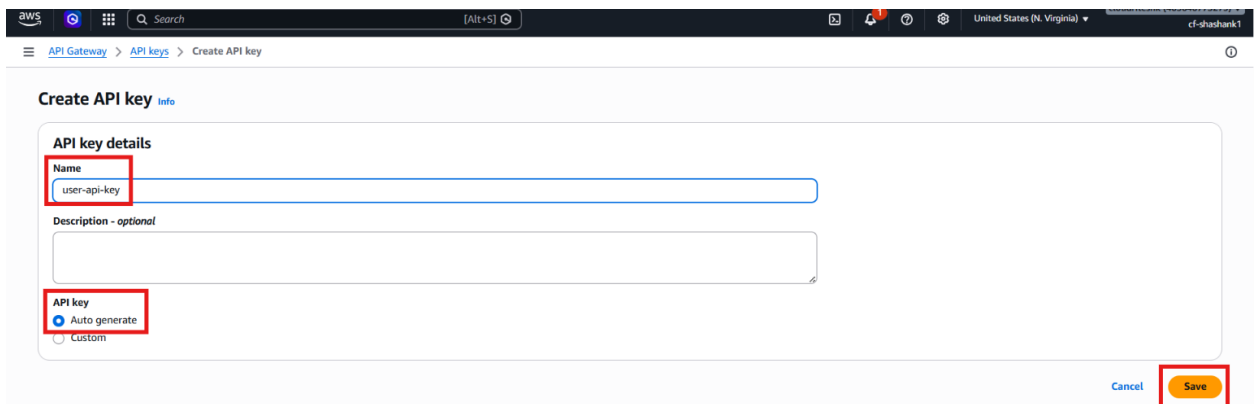
Step 2 – Create an API Key

- Open **API Gateway**.
- Click **API keys**.

- Click **Create API key**.



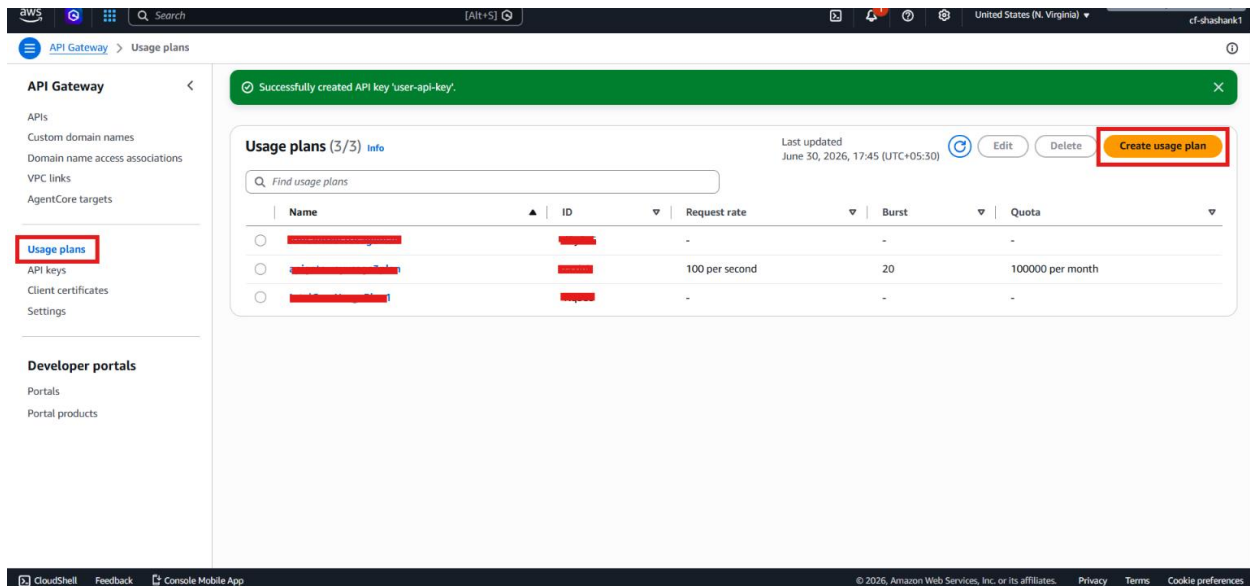
- Enter the key name: **user-api-key**
- Choose **Auto Generate**.
- Click **Save**.



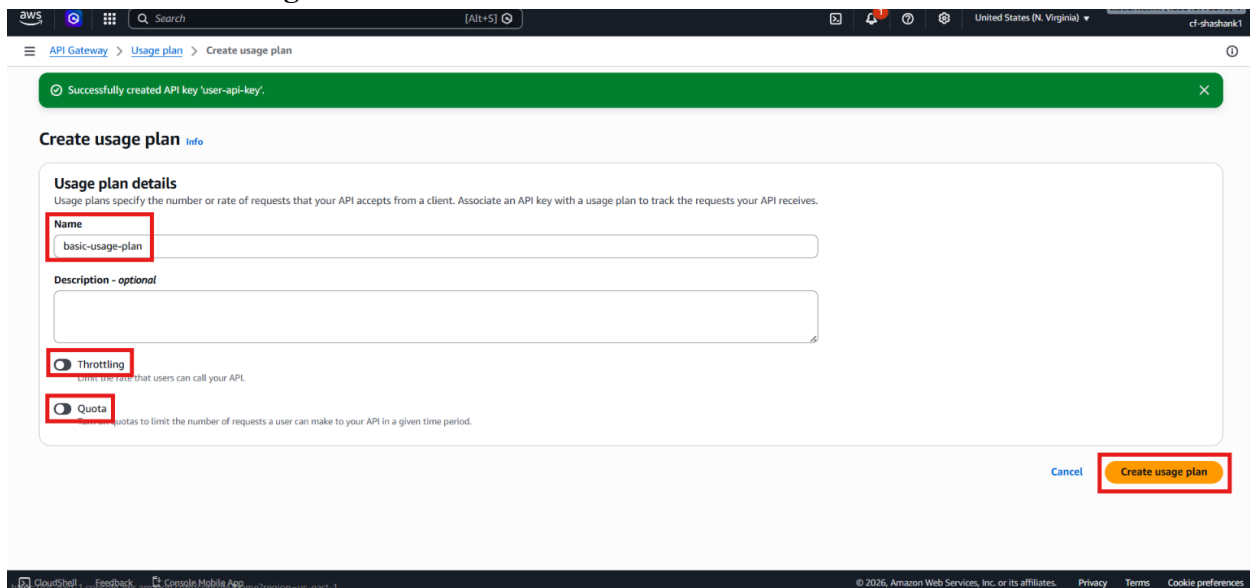
- Copy the generated key and keep it for later use in **Postman**.

Step 3 – Create a Usage Plan

- Open **API Gateway**.
- Click **Usage plans**.
- Click **Create**.



- Enter the usage plan name: basic-usage-plan
- Keep **Throttling** turned off.
- Keep **Quota** turned off for the initial test.
- Click **Create Usage Plan**.



Step 4 – Attach the REST API Stage to the Usage Plan

- Open the **basic-usage-plan**.
- Go to **Associated APIs**.
- Click **Add API**.

API Gateway > Usage plans > basic-usage-plan

basic-usage-plan

Last updated June 30, 2026, 17:48 (UTC+05:30) Actions Export usage data

Usage plan details

Usage plan ID -

Description -

AWS Marketplace product code -

Rate -

Burst -

Quota -

Associated stages Associated API keys Tags

Associated stages (0) info Edit Remove Add stage

API	Stage	Method throttling
No stages		

You don't have any stages.

Add API stage

- Select your **REST API**.
- Select the stage **Dev**.
- Save the changes.

API Gateway > Usage plans > basic-usage-plan > Associate stage

Associate stage info

Stage details

API UserRestAPI

Stage dev

You must select an API before selecting a stage.

► Method-level throttling - optional

Cancel Add to usage plan

Step 5 – Attach the API Key to the Usage Plan

- Inside the usage plan, go to **Associated API Keys**.
- Click **Add API Key**.

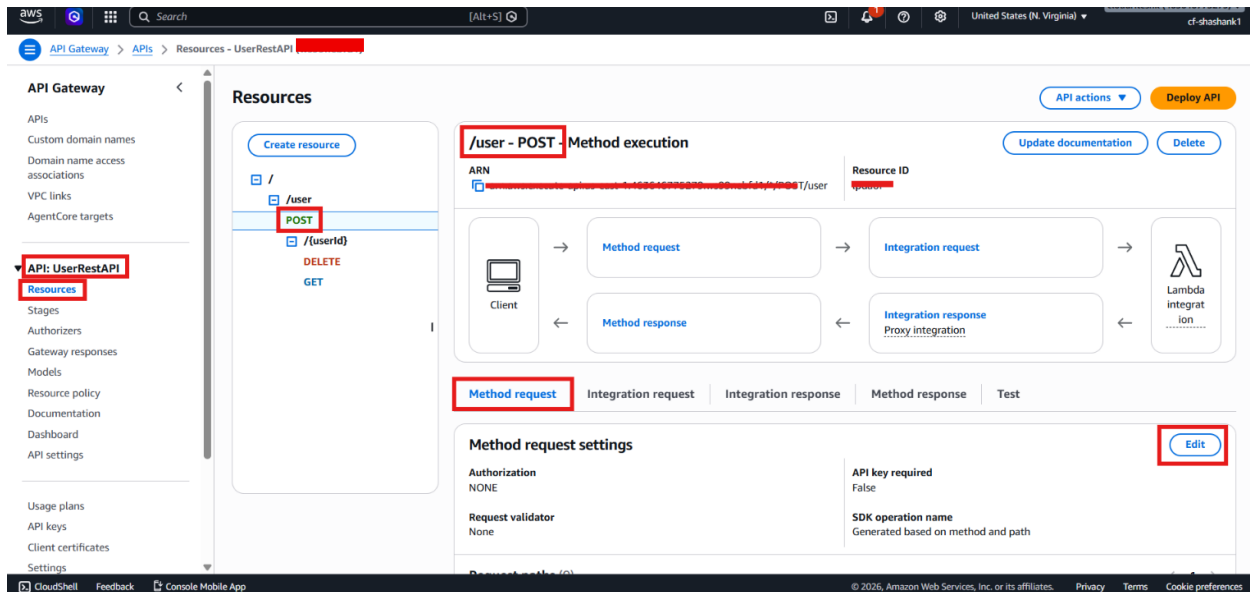
The screenshot shows the AWS API Gateway console. A green notification bar at the top states "Successfully added stage 'dev' to usage plan." The main content area is titled "basic-usage-plan" and shows "Usage plan details" with fields for Usage plan ID, Description, and AWS Marketplace product code. Below this, the "Associated API keys" tab is selected, displaying a table with columns for Name, Status, ID, API key, and Requests remaining this month. The table is currently empty, with the message "No API keys." and "This usage plan has no API keys." The "Add API key" button is highlighted in the top right corner of the API keys section.

- Select the previously created user-api-key.
- Save the changes.

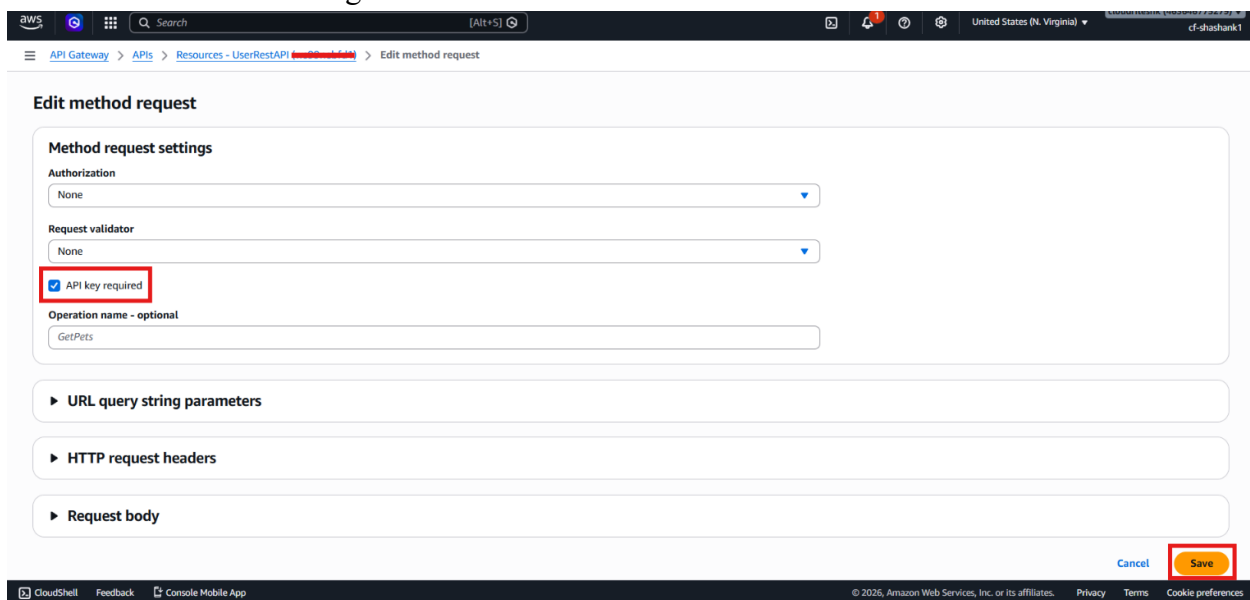
The screenshot shows the "Add API key" page in the AWS API Gateway console. A green notification bar at the top states "Successfully added stage 'dev' to usage plan." The "API key details" section shows the "Type" as "Add existing key" (selected) and "Create and add new key" (unselected). Below this, the "API key" dropdown menu is open, showing "user-api-key" as the selected option. The "Add API key" button is highlighted in the bottom right corner.

Step 6 – Require the API Key on the Method

- Open your API → Resources.
- Select the method you want to protect, such as POST.
- Click **Edit** in **Method Request** tab.

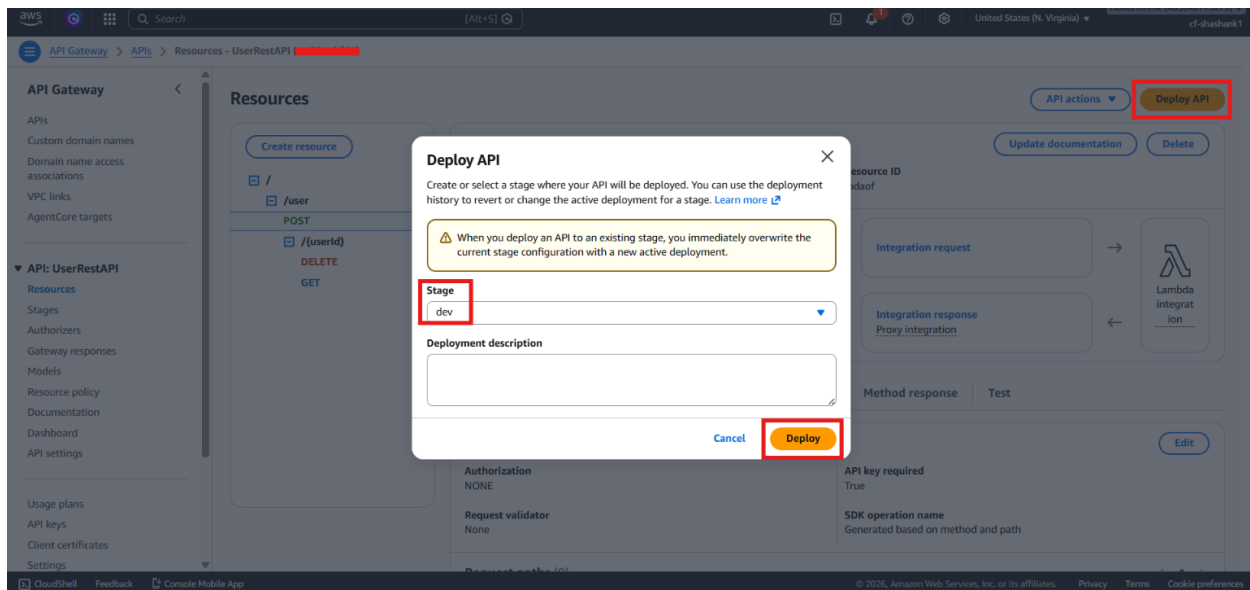


- Set API Key Required to True.
- Save the method settings.



Step 7 – Deploy the API Again

- Click **Deploy API**.
- Select the existing stage **Dev**.
- Click **Deploy**.



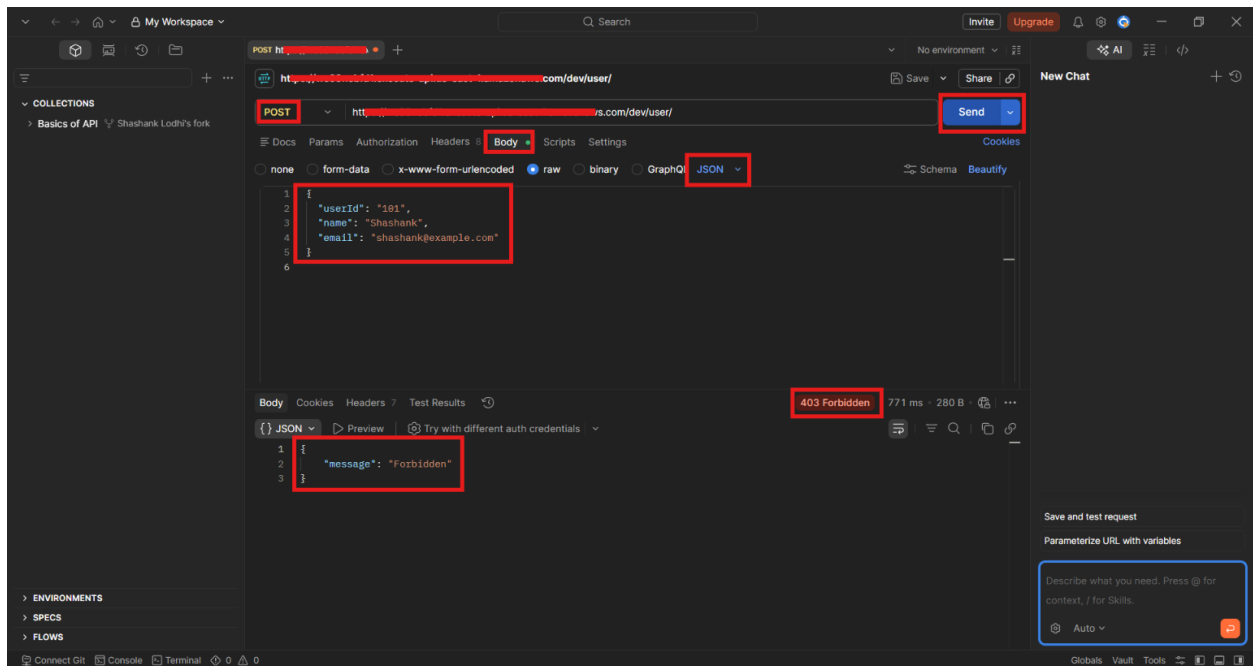
- Wait for the deployment to finish before testing.

Step 8 – Test Without the API Key

- Open **Postman**.
- Create a new **POST** request.
- Paste the invoke URL and append `/user`.
- Add the request body JSON.

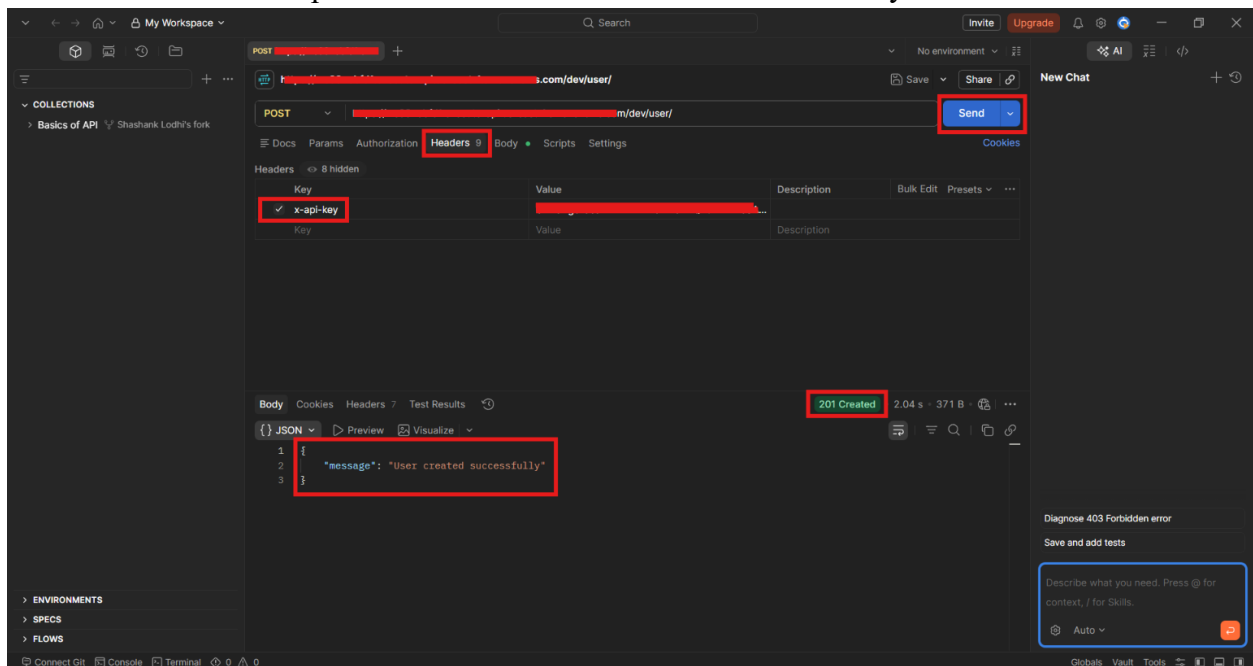
```
{
  "userId": "101",
  "name": "Shashank",
  "email": "shashank@example.com"
}
```

- Send the request without any API key header.
- Confirm that the response is **403 Forbidden**.



Step 9 – Add the API Key Header in Postman

- In **Postman**, open the **Headers** tab.
- Add the header key: `x-api-key`
- Paste the generated API key as the header value.
- Send the same POST request again.
- Confirm that the response shows the user was created successfully.



- Open **Amazon DynamoDB** and verify that the record was inserted.

The screenshot shows the AWS Management Console for the 'UsersTable' in DynamoDB. The left sidebar contains navigation links for 'DynamoDB' and 'DAX'. The main content area is titled 'UsersTable' and includes a 'Scan or query items' section. In this section, the 'Scan' button is selected, and the 'Run' button is highlighted with a red box. Below the scan results, a green banner indicates 'Completed - Items returned: 1 - Items scanned: 1 - Efficiency: 100% - RCUs consumed: 2'. The 'Table: UsersTable - Items returned (1)' section shows a single item with the following details: user id (String), email, and name. The item is highlighted with a red box, showing the user ID '101', email 'shashank@...', and name 'Shashank'.

Step 10 – Test the Usage Plan Quota

- Return to the usage plan.
- Click **Edit usage plan**.

The screenshot shows the AWS Management Console for the 'API Gateway' service, specifically the 'Usage plans' page. The left sidebar contains navigation links for 'API Gateway' and 'Developer portals'. The main content area is titled 'Usage plans (4/4)' and includes a table of usage plans. The 'Edit' button is highlighted with a red box. The table shows four usage plans, with the 'basic-usage-plan' selected and highlighted with a red box. The 'basic-usage-plan' has a quota of 100000 per month.

Name	ID	Request rate	Burst	Quota
basic-usage-plan		100 per second	20	100000 per month

- Enable **Quota**.
- Set the quota to **1 request per day**.
- Save the changes.

Edit usage plan

Usage plan details
Usage plans specify the number or rate of requests that your API accepts from a client. Associate an API key with a usage plan to track the requests your API receives.

Name
basic-usage-plan

Description - optional

☐ **Throttling**
Limit the rate that users can call your API.

☒ **Quota**
Set quotas to limit the number of requests a user can make to your API in a given time period.

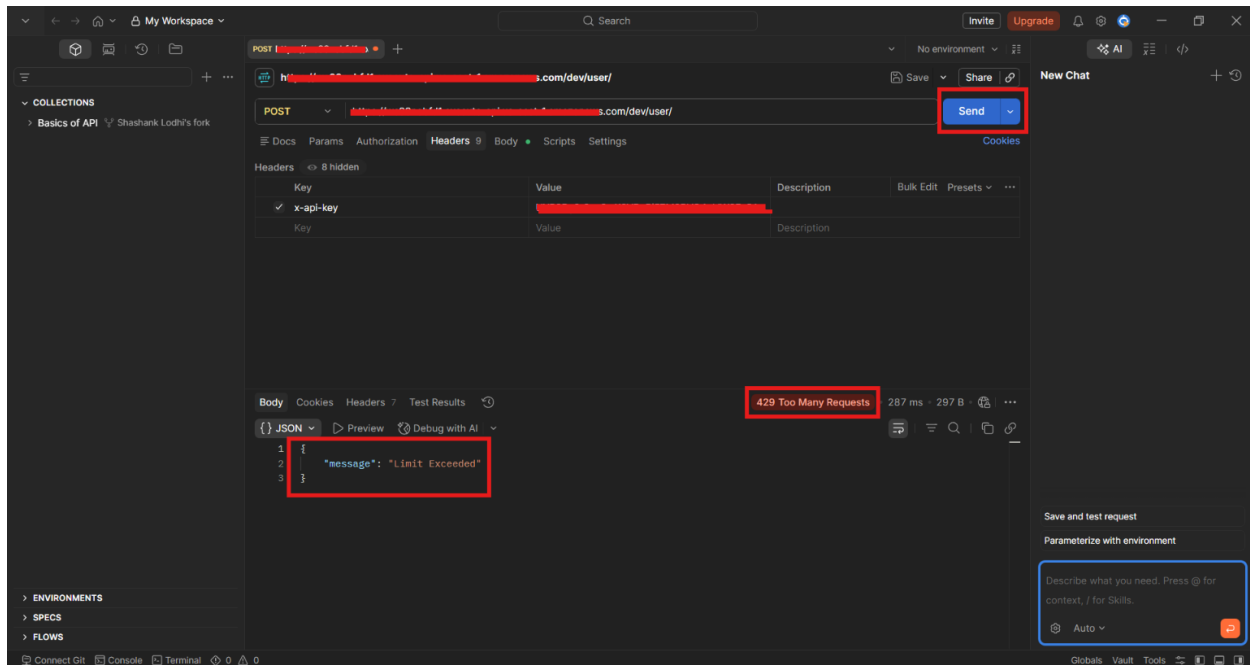
Requests
Enter the total number of requests that a user can make in the time period you select in the dropdown list.

1 Per day

Cancel Save changes

⚠ Changing the settings of your usage plan will take effect immediately. This might affect how API callers invoke your API. [Learn more](#)

- Deploy the API again.
- Send another request from **Postman**.
- Confirm that the response becomes **429 Too Many Requests** or **message limit exceeded**.



Verification

- The API key was created successfully.
- The usage plan was created successfully.
- The REST API stage **Dev** was attached to the usage plan.
- The API key was attached to the usage plan.
- The POST method was set to require an API key.
- Requests without x-api-key returned **403 Forbidden**.

- Requests with the valid API key reached **Lambda** successfully.
- The quota test returned **429 Too Many Requests** after the limit was reached.